

MongoDB Queries: Spotify Playlists

Each question is shown in the blue box, followed by its answer. Commands are for mongosh. All queries use the sample data inserted in Question 1, so run Question 1 first.

Question 1

Question 1. Insert at least 5 documents into a MongoDB collection called *playlists*, where each document represents a Spotify playlist with fields: *name* (string), *likes* (number), *genre* (string), and *followers* (number).

Answer: use `insertMany()` to add 8 playlists at once (more than the required 5, so the later queries have useful results).

```
use spotify_clone
```

```
db.playlists.insertMany([
  { name: "Workout Beats", likes: 2500, genre: "Hip-Hop", followers: 8200 },
  { name: "Chill Vibes", likes: 1800, genre: "Lo-Fi", followers: 3200 },
  { name: "Top Pop Hits", likes: 4200, genre: "Pop", followers: 15000 },
  { name: "Bollywood Party", likes: 950, genre: "Bollywood", followers: 6100 },
  { name: "Rap Caviar Mix", likes: 3100, genre: "Hip-Hop", followers: 12500 },
  { name: "Indie Sunrise", likes: 400, genre: "Indie", followers: 700 },
  { name: "Pop Classics", likes: 1200, genre: "Pop", followers: 5000 },
  { name: "Late Night Jazz", likes: 650, genre: "Jazz", followers: 900 }
])
```

Output (ObjectIds will differ; abbreviated here):

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('66f4a1...'),
    '1': ObjectId('66f4a1...'),
    ...
    '7': ObjectId('66f4a1...')
  }
}
```

Data in the collection:

name	likes	genre	followers
Workout Beats	2500	Hip-Hop	8200
Chill Vibes	1800	Lo-Fi	3200
Top Pop Hits	4200	Pop	15000
Bollywood Party	950	Bollywood	6100
Rap Caviar Mix	3100	Hip-Hop	12500
Indie Sunrise	400	Indie	700
Pop Classics	1200	Pop	5000
Late Night Jazz	650	Jazz	900

Data types: name and genre are Strings, likes and followers are Numbers.

Question 2

Question 2. Write a MongoDB query using the `$gt` operator to find all playlists with more than 1000 followers in the *playlists* collection and display only the name and followers fields. Hint: Use projection to limit the fields returned.

Answer: the first argument is the filter using `$gt` (greater than). The second argument is the projection, where `1` includes a field and `_id: 0` hides the default `_id`.

```
db.playlists.find(
  { followers: { $gt: 1000 } },
  { name: 1, followers: 1, _id: 0 }
)
```

Output:

```
[
  { name: 'Workout Beats', followers: 8200 },
  { name: 'Chill Vibes', followers: 3200 },
  { name: 'Top Pop Hits', followers: 15000 },
  { name: 'Bollywood Party', followers: 6100 },
  { name: 'Rap Caviar Mix', followers: 12500 },
  { name: 'Pop Classics', followers: 5000 }
]
```

Indie Sunrise (700) and Late Night Jazz (900) are not shown because they have 1000 followers or fewer.

Question 3

Question 3. Use the `$in` operator to find all playlists where the genre is either 'Pop' or 'Hip-Hop' and sort the results by likes in descending order.

Answer: `$in` matches any value in the given array. `sort({ likes: -1 })` sorts from highest to lowest (-1 means descending, 1 means ascending).

```
db.playlists.find(
  { genre: { $in: ["Pop", "Hip-Hop"] } },
  { _id: 0 }
).sort({ likes: -1 })
```

The `{ _id: 0 }` projection is only there to keep the output short. It is optional.

Output:

```
[
  { name: 'Top Pop Hits', likes: 4200, genre: 'Pop', followers: 15000 },
  { name: 'Rap Caviar Mix', likes: 3100, genre: 'Hip-Hop', followers: 12500 },
  { name: 'Workout Beats', likes: 2500, genre: 'Hip-Hop', followers: 8200 },
  { name: 'Pop Classics', likes: 1200, genre: 'Pop', followers: 5000 }
]
```

Question 4

Question 4. Query the `playlists` collection to find playlists with likes between 500 and 2000 (inclusive) using `$gte` and `$lte`, then limit the output to the top 3 playlists sorted by followers.

Answer: put both `$gte` and `$lte` on the same field to make a range. Sort by followers in descending order so the "top" playlists (most followers) come first, then use `limit(3)`.

```
db.playlists.find(
  { likes: { $gte: 500, $lte: 2000 } },
  { _id: 0 }
).sort({ followers: -1 }).limit(3)
```

Playlists matching the range 500 to 2000 likes (before sorting and limiting): Chill Vibes (1800), Bollywood Party (950), Pop Classics (1200) and Late Night Jazz (650).

Output (top 3 by followers):

```
[
  { name: 'Bollywood Party', likes: 950, genre: 'Bollywood', followers: 6100 },
  { name: 'Pop Classics', likes: 1200, genre: 'Pop', followers: 5000 },
  { name: 'Chill Vibes', likes: 1800, genre: 'Lo-Fi', followers: 3200 }
]
```

]

Late Night Jazz (900 followers) matched the likes range but was cut off by `limit(3)`. MongoDB applies the sort first and then the limit, so the limit keeps the 3 playlists with the most followers.

Question 5

Question 5. Suppose you want to recommend trending playlists like Spotify does. Write a MongoDB query that finds all playlists with more than 5000 followers and at least 1000 likes, but only show their name and genre. Explain in one line how you combined multiple query operators in your filter.

Answer:

```
db.playlists.find(
  { followers: { $gt: 5000 }, likes: { $gte: 1000 } },
  { name: 1, genre: 1, _id: 0 }
)
```

Output:

```
[
  { name: 'Workout Beats', genre: 'Hip-Hop' },
  { name: 'Top Pop Hits', genre: 'Pop' },
  { name: 'Rap Caviar Mix', genre: 'Hip-Hop' }
]
```

One-line explanation: I put `$gt` on followers and `$gte` on likes as two conditions in the same filter document, and MongoDB treats separate fields in one filter as an implicit AND, so a playlist must satisfy both.

Why the others are excluded: Pop Classics has exactly 5000 followers (not more than 5000), Bollywood Party has only 950 likes, and Chill Vibes has only 3200 followers.

Same filter written with an explicit `$and`:

```
db.playlists.find(
  { $and: [ { followers: { $gt: 5000 } }, { likes: { $gte: 1000 } } ] },
  { name: 1, genre: 1, _id: 0 }
)
```

Operator Summary

Operator	Meaning
<code>\$gt</code>	Greater than
<code>\$gte</code>	Greater than or equal to
<code>\$lte</code>	Less than or equal to
<code>\$in</code>	Matches any value in a given array
<code>sort({ field: -1 })</code>	Sort descending (use 1 for ascending)
<code>limit(n)</code>	Return at most n documents